

setup repository (repo)

install git

macos and linux come with git, but if you have windows, first you'll need to install it from the [official website](#).

create local git repo

git is a local software engine, specifically a distributed version control system, that operates locally to track any changes in your source code and files over time. git handles the mechanics of merging code allowing for multiple developers to work on the same codebase simultaneously without overwriting each other's work.

create a directory (folder) for your repo

you can do this manually, or you can open a terminal and move into the directory you want your repo to live in by typing:

```
cd path\to\parent\directory
```

- **cd**: change directory
 - if you're in a parent directory and want to be in one of its children or grandchildren then you can begin your path with

```
“./child/grandchild”
```

- **./**: inserts the path to the directory your terminal is currently in

then make the directory for your repo

```
mkdir repo-directory-name
```

initialize your repo

if you manually created your directory,

1. right click
2. press **copy as path**

3. open terminal and move into that directory, type:

```
cd your\directory\path
```

now everyone can initialize by running the command:

```
git init
```

if this worked you should see a directory named `.git` in your repo directory (if not, click

```
view>show>hidden items
```

and it should be there).

DO NOT TOUCH OR EDIT ANYTHING IN THIS FOLDER

add necessary directories and files

here I'll explain using the minimal set, some projects might require other directories for setup. in your terminal, type the command corresponding to your os:

windows

```
new-item -itemtype file src/.gitkeep, results/.gitkeep, tests/.gitkeep, .gitignore, README.md -force
```

breakdown of the command:

- `new-item`: creates new item(s)
 - separate multiple args with commas
- `-itemtype file`: indicates the new items we want are files
 - If only making directories, replace file with directory
- `-force`: automatically make any parent folders that don't exist

macos/linux

```
mkdir -p src results tests && touch src/.gitkeep results/.gitkeep tests/.gitkeep .gitignore README.md
```

breakdown of the command(s):

- `mkdir`: (only) creates new directorie(s)
- `touch`: (only) creates new empty file(s)
- for both `mkdir` and `touch`:
 - separate multiple args with spaces
 - if directory names have spaces, use quotations (e.g. `mkdir "new directory"`)
- `-p`: automatically make any parent folders that don't exist

- not necessary here per say, but does make the command idempotent (safe to run repeatedly without breaking)
- `&&`: chains the commands. says “if directories are created successfully, immediately run the next command”

create a virtual environment

a **virtual environment (venv)** is a self-contained location where you can maintain separate and isolated environments for your python projects. so you can manage dependencies, versions and packages without conflict with other projects (e.g. certain projects may require different python or external library versions)

in this step we will be creating a venv directory in your repo which contains an isolated copy of python.

windows

in the terminal, to use the version of python your system currently has, run the command:

```
python -m venv venv
```

to use a specific version (e.g. `3.##`) of python in the venv, instead run:

```
py -#.# -m venv venv
```

breakdown of the command:

- `python` or `py`: invokes the python interpreter program
- `-m`: this is the module flag, it tells python to execute a built-in module
- the first `venv`: specifies we want to run the venv module
- the second `venv`: the name of the directory you want to create

macos/linux

in the terminal, to use the version of python your system currently has, run the command:

```
python3 -m venv venv
```

to use a specific version (e.g. `3.##`) of python in the venv, instead run:

```
python3.## -m venv venv
```

breakdown of the command:

- `python3`: invokes the python interpreter program
- `-m`: this is the module flag, it tells python to execute a built-in module
- the first `venv`: specifies we want to run the venv module
- the second `venv`: the name of the directory you want to create

activate your virtual environment

once you activate your venv (i.e. you “move your terminal into” this venv), you will see `(venv)` appear at the beginning of your terminal prompt.

windows

in your terminal, run the following command:

```
.\env\scripts\activate.ps1
```

- If you get the error: `Execution_Policies_Requirement_Error` or “script execution is disabled on this system.”, then run this quick fix command:

```
set-executionpolicy -executionpolicy remotesigned -scope process
```

This should enable scripts just in the current terminal without changing your computers global security settings.

Then try to run the activation command again:

```
.\env\scripts\activate.ps1
```

macos/linux

in your terminal, run the following command:

```
source venv/bin/activate
```

breakdown of the command:

- `source`: a shell command which reads and executes settings inside a script file directly into the current terminal window instead of as a background process

add virtual environment to .gitignore

as a rule, you should never push your venv folder to github, it contains a ton of local files unique to your operating system.

open your .gitignore file and add this line to the top:

```
venv/
```

commit

a **commit** is like a checkpoint, or a snapshot of your projects files at a specific moment of time. it records what changes were made, files were added/removed, and metadata like the author's name, a timestamp and a descriptive message of why/what changes were made.

the usual procedure for any commits is to first check what changes you have made to your repo. Since we are planning to commit to all the changes we made, we don't necessarily need to do this right now, but its good to get used to doing it every time so you can ensure you only commit the changes you need to.

to check your changes, run the command:

```
git status
```

the terminal will then list all the changes you made to your repo. next you'll choose what changes you want to commit.

to select specific files, run the command:

```
git add your_file your_other_file
```

to select all changes, run the command:

```
git add .
```

then you can commit by running the command:

```
git commit -m "initial setup"
```

connect local repo to github

github is a cloud-based hosting platform that stores git projects, so it lives on the internet (i.e. remote servers) instead of on your local computer. github acts as an “official” version of your project.

why do you need github if git already allows for collaboration? github provides the infrastructure and interface to make global scale collaboration feasible.

git does allow for peer-to-peer collaboration, but doing so without connection to github requires you to manually set up secure network access to your collaborators physical computers.

github also introduces code review and pull requests (pr) and additional project management tools like issues, projects, and wikis. It also allows you to manage access control and security, so you can choose what parts of a project are shared with what collaborators and restrict who is allowed to push code.

create a blank repo on github

go to [github](#) and sign in

in the top right, click the + icon and select [New repository](#)

[give it a name](#)

make sure you leave ["Add a README file"](#), ["Add .gitignore"](#), and ["Choose a license"](#) unchecked since we already created those

click [Create repository](#)

copy connection lines

once the repo is created, GitHub will show a quick-setup page with a section called ["...or push an existing repository from the command line"](#).

it will look exactly like this (with your github username instead of YOUR-USERNAME):

```
git remote add origin https://github.com/YOUR-USERNAME.git
```

```
git branch -M main
```

```
git push -u origin main
```

run the copied commands in your terminal

- `git remote add origin <URL>`: creates a link between their local computer and github. it names this connection origin (the industry-standard name for your primary online repo)
- `git branch -M main`: ensures their default branch is named main instead of master (matching modern github standards)
- `git push -u origin main`: uploads ("pushes") all their files up to github for the first time.
 - `-u`: flag that remembers this choice so that next time, you only have to type `git push` to upload new changes